



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)

НАПРАВЛЕНИЕ ПОДГОТОВКИ 09.03.03 Прикладная информатика

БАКАЛАВРСКАЯ ПРОГРАММА 09.03.03/02 Программно-технические средства
информатизации

ОТЧЕТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

Тип практики Эксплуатационная

Название
предприятия НУК ИУ МГТУ им. Н.Э. Баумана

Студент ИУ6-64Б

Д.О. Кошенков

(Подпись, дата)

(И.О. Фамилия)

Руководитель практики
от МГТУ им.
Н.Э.Баумана

С.С. Данилюк

(Подпись, дата)

(И.О. Фамилия)

Оценка _____

2025 г.

З А Д А Н И Е на производственную практику

по теме Прохождение курса VK Education Practice

Студент группы ИУ6-64Б

Кошенков Дмитрий Олегович
(Фамилия, имя, отчество)

Направление подготовки 09.03.03 Прикладная информатика

Бакалаврская программа 09.03.03_02 Программно-технические средства информатизации

Тип практики Эксплуатационная практика

Название предприятия НУК ИУ МГТУ им. Н.Э. Баумана

Техническое задание Прохождение курса “Алгоритмы и Структуры Данных” и разработка
сервиса сегментирования пользователей

Оформление отчета по практике:

Отчет на 15-25 листах формата А4.

Перечень графического (иллюстративного) материала (чертежи, плакаты, слайды и т.п.)

нет

Дата выдачи задания « 27 » июня 2025 г.

Руководитель практики
от МГТУ им. Н.Э.Баумана

Студент

27.06.2025
(Подпись, дата)

С.С. Данилюк
(И.О. Фамилия)

27.06.2025
(Подпись, дата)

Д.О. Кошенков
(И.О. Фамилия)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Архитектура системы	5
1.1 Обзор архитектурных принципов	6
1.2 Назначение и взаимодействие компонентов	6
2 Схема потоков данных	7
3 Модель данных	8
3.1 Структура Experiment	9
3.2 Правила таргетинга	9
3.3 Варианты и распределение	10
3.4 Списки принудительного включения/исключения	10
4 Реализация серверной части	10
5 Алгоритм принятия решений	11
5.1 Порядок обработки правил	12
5.2 Механизм распределения пользователей	12
6 Асинхронное обновление	13
6.1 Надёжная доставка	14
6.2 Роль брокера	14
6.3 Восстановление состояний	14
7 Клиентская библиотека	14
7.1 Логика инициализации: холодный старт и загрузка снимка	15
7.2 Кэширование конфигурации	15
7.3 Обновления кэша	15
ЗАКЛЮЧЕНИЕ	15
ПРИЛОЖЕНИЕ А КУРС «АЛГОРИТМЫ И СТРУКТУРЫ ДАННЫХ»	17
ПРИЛОЖЕНИЕ Б РАЗРАБОТКА СЕРВИСА СЕГМЕНТИРОВАНИЯ	19
ПРИЛОЖЕНИЕ В ПРАКТИКА В VK EDUCATION	21

ВВЕДЕНИЕ

Проект представляет собой реализацию платформы для проведения А/В-экспериментов и динамического сегментирования пользовательской базы. Основная проблема, решаемая системой, — это необходимость в управляемом и измеримом развертывании новой функциональности, что позволяет принимать продуктовые решения на основе количественных данных. Платформа обеспечивает механизм для контролируемого включения функций, таких как голосовые сообщения, скидки на подписку или интеграция новых алгоритмов, для заданных целевых аудиторий.

В дополнение к разработке и внедрению системы в рамках работы был успешно пройден курс «Алгоритмы и структуры данных». В ходе курса изучены и проработаны структуры данных (массивы, списки, очереди, стеки, деревья, хеш-таблицы, графы) и решены практические задания, что позволило повысить качество архитектурных решений и реализовать эффективные алгоритмические части системы; сертификат о прохождении курса прилагается в приложении.

Ключевые технические задачи, решённые в рамках проекта:

- разработка единой точки входа для управления конфигурациями экспериментов;
- создание высокопроизводительной клиентской библиотеки, которая интегрируется в сервисы-потребители;
- построение асинхронного конвейера для распространения обновлений.

Архитектура системы разработана с учётом следующих нефункциональных требований:

- отказоустойчивость и автономность;
 - алгоритм принятия решения должен выполняться в рамках выполнения проекта
- был успешно пройден курс «Алгоритмы и структуры данных». В ходе курса изучены и проработаны основные структуры данных (массивы, списки, очереди, стеки, деревья, хеш-таблицы, графы) и решены практические задания; сертификат о прохождении курса прилагается в приложении.
- лняться за время, измеряемое в микросекундах, без синхронных

сетевых вызовов; – масштабируемость: архитектура должна поддерживать горизонтальное масштабирование;

– асинхронное обновление: обновления конфигураций должны распространяться по системе асинхронно, не блокируя основной процесс принятия решений.

1 Архитектура системы

1.1 Обзор архитектурных принципов

Система спроектирована как асинхронная, событийно-ориентированная архитектура, состоящая из независимых микросервисов. Такой подход обеспечивает слабую связанность компонентов, что позволяет им развиваться и масштабироваться независимо друг от друга. Ключевым архитектурным решением является разделение системы на два логических потока: управляющий поток и рабочий поток. Управляющий поток отвечает за медленные, но критически важные операции по изменению конфигурации экспериментов, где приоритетом является надёжность. Рабочий поток отвечает за сверхбыстрые локальные операции по принятию решений для пользователей, где приоритетом является производительность и отказоустойчивость.

1.2 Назначение и взаимодействие компонентов

Центральный API функционирует как шлюз для всех изменений конфигурации и является единственным компонентом с правом записи в основное хранилище данных. Он предоставляет синхронный REST API для внешних систем управления и инициирует асинхронный процесс распространения обновлений;

Реляционная база данных является персистентным хранилищем и единственным источником истины для всех конфигураций экспериментов. Использование транзакционной СУБД критически важно для реализации паттерна Transactional Outbox, обеспечивая ACID-гарантии при одновременном изменении состояния системы и создании события об этом изменении.

Обработчик outbox решает проблему двойной записи, возникающую при необходимости атомарно изменить данные в базе и отправить сообщение в брокер. Он асинхронно опрашивает специальную таблицу outbox, извлекает неотправленные события и надёжно публикует их в шину сообщений, гарантируя, что ни одно изменение конфигурации не будет потеряно.

Шина сообщений, реализованная как персистентный партиционированный лог, служит для асинхронной и надёжной доставки событий об

изменениях до всех подписчиков. Такая архитектура позволяет клиентской библиотеке быть временно оффлайн и наверстывать пропущенные обновления при восстановлении соединения, а также полностью отделяет жизненный цикл сервисов-писателей от сервисов-читателей.

Данный механизм состоит из двух частей:

1) генератор снимков, который периодически создаёт полные снимки всех активных конфигураций, и объектное хранилище, где эти снимки сохраняются, это решает проблему холодного старта, позволяя новым или перезапущенным экземплярам клиентской библиотеки мгновенно загрузить актуальное состояние вместо того, чтобы обрабатывать длинную историю предыдущих изменений из шины сообщений.

2) клиентская библиотека является ключевым элементом рабочего потока и представляет собой встраиваемую библиотеку; она содержит движок для принятия решений, который работает исключительно в памяти процесса сервиса-потребителя и обеспечивает принятие решения за микросекунды, а также автономно управляет своим состоянием, получая обновления асинхронно и обладая логикой самовосстановления через снимки состояния.

2 Схема потоков данных

Управляющий поток:

- запрос на изменение конфигурации поступает в центральный API;
- центральный API начинает транзакцию в базе данных, в рамках которой обновляет таблицу experiments и создаёт запись в таблице outbox;
- обработчик outbox в фоновом режиме извлекает запись и публикует соответствующее событие в шину сообщений;
- экземпляры клиентской библиотеки получают событие и атомарно обновляют свой оперативный кэш.

Рабочий поток:

- сервис-потребитель вызывает метод Decide у клиентской библиотеки, передавая идентификатор пользователя и его атрибуты;
- клиентская библиотека выполняет алгоритм сегментации, используя данные из оперативной памяти, и немедленно возвращает результат;
- после принятия решения клиентская библиотека асинхронно отправляет событие о назначении пользователя в шину сообщений для сбора аналитики, не блокируя основной поток выполнения.

3 Модель данных

Модели данных определяют основной контракт и структуру конфигурации A/B-экспериментов в системе. Все компоненты, от API до клиентской библиотеки, оперируют этими унифицированными сущностями, которые персистентно хранятся в реляционной базе данных с использованием типа `jsonb` для гибкости.

3.1 Структура Experiment

Основной сущностью является `Experiment`, агрегирующая всю информацию, необходимую для проведения теста:

- `id` является уникальным неизменяемым идентификатором эксперимента;
- `layerid` определяет слой для управления взаимоисключением экспериментов, гарантируя, что один пользователь не может одновременно участвовать в нескольких тестах в одном слое;
- `configversion` представляет собой `UUIDv7`, обеспечивающий строгий хронологический порядок и идемпотентность применения обновлений конфигурации;
- `salt` является уникальной строкой для каждого эксперимента, используемой в функции хеширования для обеспечения статистической независимости распределений пользователей между разными тестами;
- `status` определяет текущее состояние жизненного цикла эксперимента, например черновик, активен или завершён.

3.2 Правила таргетинга

Правила таргетинга определяют условия, которым должен удовлетворять пользователь для попадания в целевую аудиторию эксперимента:

- `attribute` указывает на атрибут из контекста пользователя, например `app_version`, `country` или `subscription_type`;
- `operator` задаёт логическую операцию сравнения, такую как `equals`, `greater_than`, `in_list` или `version_greater_than`;
- `value` содержит значение для сравнения с атрибутом пользователя.

Пользователь считается частью целевой аудитории только при выполнении всех правил из массива правил таргетинга, которые объединены между собой неявным логическим оператором и.

3.3 Варианты и распределение

Распределение пользователей по контрольным и тестовым группам определяется через сущность вариант:

- name является уникальным в рамках эксперимента строковым идентификатором варианта, например control или treatment_a;
- диапазон бакетов определяет непрерывный диапазон в числовом пространстве от 0 до 999, соответствующий данному варианту;
- сумма длин диапазонов всех вариантов в активном эксперименте определяет общий процент пользователей, участвующих в тесте.

3.4 Списки принудительного включения/исключения

Данная структура обеспечивает механизм для ручного управления участием пользователей, который имеет наивысшим приоритетом и выполняется до проверки правил таргетинга и процентного распределения:

- принудительное включение является отображением, где ключ — это имя варианта, а значение — массив идентификаторов пользователей, которые принудительно назначаются в этот вариант;
- принудительное исключение является массивом идентификаторов пользователей, которые принудительно исключаются из участия в данном эксперименте.

4 Реализация серверной части

Сервис центральный API является ядром управляющего потока; он реализован на Go с использованием фреймворка chi для маршрутизации HTTP-запросов и предоставляет интерфейс для управления жизненным циклом экспериментов и обеспечения атомарности изменений в источнике истины.

Операции создания/чтения/обновления/удаления для управления экспериментами:

- `post /experiments`: создаёт новый эксперимент; при создании автоматически генерируются уникальные `id` и соль, а также `configversion` (UUIDv7), который фиксирует время создания; эксперимент сохраняется в базу данных вместе с соответствующим событием `upsert` в таблице `outbox` в рамках одной транзакции;

- `get /experiments/{experimentid}`: возвращает полную и актуальную конфигурацию эксперимента по его идентификатору;

- `put /experiments/{experimentid}`: обновляет существующий эксперимент; при каждом обновлении генерируется новый `configversion`, что обеспечивает версионирование; операция также является транзакционной и создаёт событие `upsert` в `outbox`;

- `delete /experiments/{experimentid}`: удаляет эксперимент из базы данных; эта операция создаёт событие `delete` в `outbox`, чтобы клиентские экземпляры могли удалить конфигурацию из своего кэша.

5 Алгоритм принятия решений

Алгоритм принятия решений является детерминированным и выполняется с соблюдением строгого порядка приоритетов. Эта логика реализована в клиентской библиотеке для локальных вычислений, что обеспечивает консистентность результатов по всей системе.

5.1 Порядок обработки правил

Процесс определения участия пользователя в эксперименте представляет собой последовательность проверок, где отрицательный результат на любом из шагов немедленно прекращает дальнейшую обработку для данного эксперимента:

- анализируется статус эксперимента; вычисление продолжается только для экспериментов со статусом активен, у которых не истекло время завершения;
- проверяется нахождение идентификатора пользователя в списке принудительного исключения, при совпадении пользователь исключается из эксперимента;
- проверяется нахождение идентификатора пользователя в списках принудительного включения, при совпадении пользователю назначается указанный вариант и все последующие шаги, включая таргетинг и бакетирование, пропускаются;
- если пользователь не был исключён или принудительно включён на предыдущем шаге, выполняется валидация его атрибутов на соответствие правилам таргетинга, все правила в конфигурации эксперимента объединены между собой неявным логическим оператором и, следовательно, пользователь должен удовлетворять каждому из них, в случае несоответствия хотя бы одному правилу пользователь считается не входящим в целевую аудиторию и обработка прекращается.

5.2 Механизм распределения пользователей

Пользователи, успешно прошедшие все предыдущие проверки, попадают в механизм процентного распределения для определения их варианта.

Хеширование и бакетирование с использованием xxhash:

- входной строкой для хеш-функции служит конкатенация идентификатора пользователя (userid) и уникальной соли эксперимента (salt);
- для вычисления хеша используется xxhash.sum64, выбранная за производительность;
- полученное 64-битное хеш-значение делится по модулю на количество бакетов для отображения пользователя на один из них;
- вычисленный номер бакета сопоставляется с диапазонами бакетов, определёнными для каждого варианта в конфигурации эксперимента пользователь назначается в тот вариант, в чей диапазон попадает его бакет.

Использование уникальной строки для каждого эксперимента является требованием для обеспечения статистической независимости; разные соли гарантируют, что распределение одного и того же пользователя по бакетам будет псевдослучайным и независимым от других экспериментов.

6 Асинхронное обновление

Основой архитектуры является асинхронная модель распространения изменений, которая обеспечивает слабую связанность компонентов и высокую отказоустойчивость клиентской части.

6.1 Надёжная доставка

Для устранения проблемы двойной записи применяется паттерн Transactional Outbox, в рамках одной ACID-транзакции в центральном API выполняются две операции — обновление данных в таблице experiments и вставка записи о событии в таблицу outbox, выделенный обработчик outbox периодически опрашивает эту таблицу, забирает необработанные события и гарантированно доставляет их в Kafka. После успешной отправки запись из outbox удаляется.

6.2 Роль брокера

Kafka используется как персистентная и отказоустойчивая шина сообщений, обработчик outbox публикует события об изменениях в топик ab_deltas, каждое сообщение содержит полную конфигурацию изменённого или созданного эксперимента и в качестве ключа использует experiment_id, что обеспечивает попадание всех изменений, относящихся к одному эксперименту, в одну и ту же партицию и сохраняет порядок их обработки для подписчиков.

6.3 Восстановление состояний

Для решения проблемы холодного старта реализован механизм снимков. Сервис-генератор снимков периодически считывает из postgres конфигурации всех активных экспериментов и сохраняет их в виде одного json-файла в MinIO. При запуске клиентская библиотека сначала пытается загрузить последний снимок из MinIO, и при успехе мгновенно получает актуальное состояние. В случае неудачи клиентская библиотека может использовать локально сохранённый кэш предыдущего снимка. После загрузки снимка клиентская библиотека начинает потреблять изменения из Kafka для получения последующих обновлений.

7 Клиентская библиотека

Клиентская библиотека является ключевым компонентом рабочего потока, предназначенным для встраивания в сервисы-потребители. Библиотека спроектирована для обеспечения максимальной производительности и отказоустойчивости при принятии решений.

7.1 Логика инициализации: холодный старт и загрузка снимка

Процесс инициализации клиентской библиотеки является блокирующей операцией, которая обеспечивает загрузку валидной конфигурации перед началом работы:

- при старте клиент в первую очередь пытается загрузить самый последний снимок из объектного хранилища MinIO;
- в случае недоступности MinIO клиентская библиотека пытается прочитать последнюю успешно загруженную конфигурацию из локального файлового кэша;
- после успешной загрузки данных из MinIO или локального файла они парсятся и помещаются в оперативный кэш.

Инициализация считается завершённой только после успешного заполнения оперативного кэша одним из этих способов.

7.2 Кэширование конфигурации

Вся логика принятия решений клиентской библиотеки работает с конфигурацией, полностью находящейся в оперативной памяти. Подход обеспечивает доступ к данным и позволяет выполнять алгоритм принятия решений за микросекунды, так как исключаются сетевые вызовы или операции дискового ввода-вывода в критическом пути.

7.3 Обновления кэша

Сразу после успешной инициализации клиентская библиотека запускает в фоновом режиме потребителя, при получении нового сообщения выполняются следующие действия:

- сравнивает configversion из сообщения с версией, хранящейся в кэше;
- применяет изменение к оперативному кэшу, атомарно обновляя или добавляя конфигурацию эксперимента.

ЗАКЛЮЧЕНИЕ

Фундаментом для разработки послужила углубленная теоретическая подготовка, полученная на курсе «Алгоритмы и структуры данных». В процессе обучения были освоены ключевые структуры данных, такие как массивы, списки, деревья, графы и хэш-таблицы, а также изучены базовые алгоритмы сортировки и основы динамического программирования. Успешное прохождение курса подтверждено сертификатом, который находится в Приложении А.

В результате выполнения проекта была разработана и реализована комплексная платформа для сегментирования пользователей и проведения А/В-экспериментов, полностью отвечающая исходным требованиям. Успешная реализация данного сервиса подтверждается сертификатом, представленным в Приложении Б.

Вся работа проводилась в рамках образовательной практики VK Education, что подтверждается итоговым сертификатом в Приложении В.

ПРИЛОЖЕНИЕ А
КУРС «АЛГОРИТМЫ И СТРУКТУРЫ ДАННЫХ»
Листов 1



СЕРТИФИКАТ УЧАСТНИКА КУРСА

Успешно прослушал(а) курс асинхронного формата «Алгоритмы и структуры данных»

МОДУЛИ КУРСА:

- Модуль 1: основы алгоритмов и структур данных
- Модуль 2: поиск данных и методы сортировки
- Модуль 3: хеширование и строки
- Модуль 4: деревья
- Модуль 5: продвинутые алгоритмы



ПРИЛОЖЕНИЕ Б
РАЗРАБОТКА СЕРВИСА СЕГМЕНТИРОВАНИЯ
Листов 1

Сертификат

об успешном завершении работы над практическим проектом

Сервис сегментирования пользователей

Кошенков Дмитрий

Заместитель вице-президента
по образовательным проектам VK
Анна Степанова



ПРИЛОЖЕНИЕ В
ПРАКТИКА В VK EDUCATION
Листов 1

Сертификат

об успешном прохождении онлайн-практики

VK Education Practice

Кошенков Дмитрий

Заместитель вице-президента
по образовательным проектам VK
Анна Степанова

